PROYECTO ACADÉMICO DE CIBERSEGURIDAD

Control de transferencia de archivos mediante políticas DLP

Casos de uso, arquitectura de red
y diseño del sistema

INFORME TÉCNICO

Prototipo de laboratorio Windows / Kali Linux

Lima, Perú

Octubre de 2026

Estado documentado: implementación por reglas, previa a la integración de Groq

Resumen

Se documenta un prototipo de prevención de fuga de información aplicado a un canal de carga de archivos. El laboratorio utiliza una computadora Windows como anfitrión del servicio y una máquina virtual Kali Linux como cliente de pruebas. La comunicación se realiza mediante una red Host-only y un servicio HTTP que recibe documentos ficticios en formato TXT, CSV o JSON.

El control inspecciona el texto con siete reglas, suma sus puntajes y resuelve si permite el almacenamiento, lo permite con alerta o lo bloquea. La decisión conserva una explicación basada en las reglas coincidentes y un registro de auditoría. Este informe presenta los actores, seis casos de uso, la topología de red, los componentes de software y la secuencia de procesamiento. También establece el alcance de los resultados experimentales y las limitaciones de la implementación.

Palabras clave: DLP, clasificación de información, transferencia de archivos, auditoría, laboratorio de ciberseguridad.

Contenido

Sección	Página
1. Propósito, alcance y actores	3
2. Modelo de casos de uso	4
2.1. Especificación de la transferencia de archivos	5
2.2. Consultas y obtención de ejemplos	6
2.3. Operación técnica y evaluación	7
3. Arquitectura de red	8
4. Arquitectura de software	9
5. Políticas de clasificación y decisión	10
6. Flujo de procesamiento	11
7. Secuencia y persistencia	12
8. Evaluación y lectura de resultados	13
9. Conclusiones, limitaciones y fuentes	14

1. Propósito, alcance y actores

1.1. Problema y objetivo del control

En un canal de transferencia, la recepción de un documento no basta para determinar si su almacenamiento está permitido. Un archivo puede contener información académica individual, credenciales o una clasificación interna que requiera un tratamiento distinto al de un documento público. El prototipo permite ensayar esa decisión con datos ficticios y observar sus consecuencias.

El objetivo general es implementar y evaluar un control que inspeccione el contenido antes de guardarlo en el destino del laboratorio. Se busca identificar indicadores de sensibilidad, aplicar una política reproducible y conservar evidencia suficiente para contrastar cada decisión con una clasificación de referencia.

1.2. Delimitación del escenario

El servicio Windows funciona como receptor y punto de control del canal simulado. Kali inicia las cargas desde el navegador o desde una terminal. Aunque se utilicen dos sistemas operativos, ambos pueden ejecutarse en una única computadora física: Windows aloja la máquina virtual mediante VirtualBox.

El bloqueo ocurre después de recibir los bytes en memoria y antes de escribirlos en la carpeta de destino. Por tanto, la evidencia demuestra control del almacenamiento en este servicio. No demuestra inspección de todas las transferencias de la PC ni interceptación de tráfico hacia otros destinos.

Tabla 1. Actores del sistema

Actor	Responsabilidad en el laboratorio
Participante de prueba	Seleccionar archivos ficticios, intentar transferirlos y observar la decisión del control.
Evaluador o auditor	Examinar políticas y eventos; contrastar las decisiones con la clasificación de referencia.
Operador técnico	Instalar y arrancar el servicio; modificar reglas, ejecutar pruebas y activar cambios mediante un reinicio.

Los actores representan responsabilidades y pueden ser desempeñados por una misma persona. La implementación no incorpora cuentas, inicio de sesión ni autorización por roles. Las consultas del portal son accesibles a quienes alcanzan el servicio en la red del laboratorio.

2. Modelo de casos de uso

El modelo distingue las funciones ofrecidas por el portal de las tareas que requieren terminal o edición del código. Esta separación evita atribuir a la interfaz capacidades de administración que todavía no están implementadas.

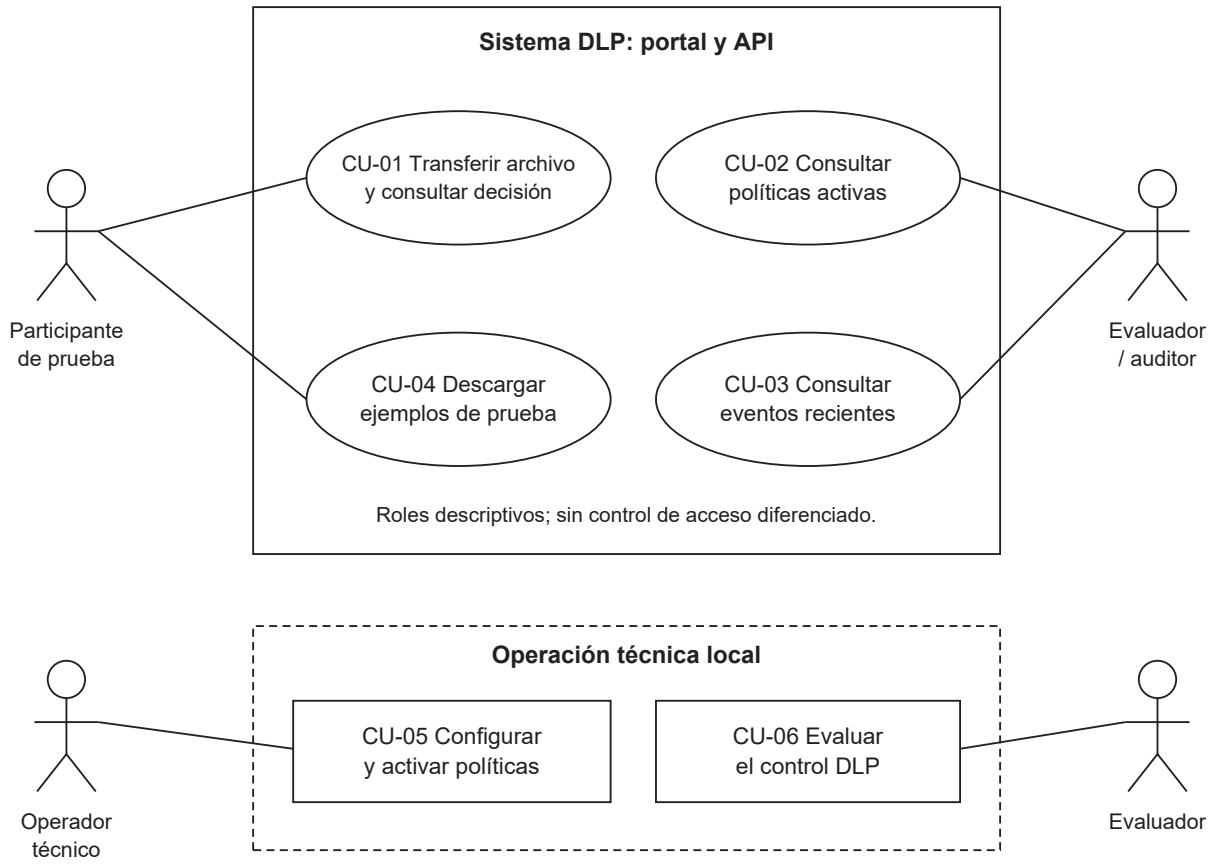


Figura 1. Casos de uso del portal y operación técnica local.

Nota. Los roles son descriptivos. Fuente: interfaz y código del prototipo [1, 2].

Tabla 2. Catálogo de casos de uso

Código	Caso de uso	Acceso
CU-01	Transferir archivo y consultar decisión	Portal / API
CU-02	Consultar políticas activas	Portal / API
CU-03	Consultar eventos recientes	Portal / API
CU-04	Descargar ejemplos de prueba	Portal
CU-05	Configurar y activar políticas	Código / terminal
CU-06	Evaluar el control DLP	Terminal

2.1. Transferencia de archivos: CU-01

Tabla 3. Especificación de CU-01

Campo	Especificación
Actor principal	Participante de prueba.
Objetivo	Solicitar una transferencia al destino controlado y conocer si su almacenamiento se permite o se bloquea.
Precondiciones	Servicio disponible y accesible; archivo ficticio presente en el equipo cliente.
Disparador	Pulsar «Analizar y transferir» o enviar una solicitud a POST /api/upload.
Entrada	Un archivo en el campo multipart denominado archivo. Extensiones admitidas: .txt, .csv y .json; texto UTF-8; máximo 1 MiB (1.048.576 bytes).
Salida	Decisión, puntaje, coincidencias, mensaje y metadatos del evento.
Postcondición	Si se permite o alerta: archivo guardado y evento registrado. Si se bloquea: evento registrado y archivo sin guardar.

Flujo principal

1. El cliente selecciona el archivo y envía la carga al servicio HTTP.

2. Multer comprueba la extensión y los límites; el servidor verifica la presencia del archivo y decodifica el texto UTF-8.

3. El motor prepara el contenido y aplica las reglas. Para CSV añade una representación de columnas y valores; para TXT y JSON examina el texto recibido.

4. Se suman los puntos de las reglas coincidentes y se calcula la decisión.

5. Si corresponde, se guarda el archivo con un nombre normalizado y un identificador UUID. Después se agrega el evento de auditoría.

6. La API devuelve HTTP 200 o 403. El navegador presenta el resultado y actualiza la consulta de eventos.

Alternativas y errores

La ausencia de archivo, una extensión no admitida, el exceso de tamaño o un texto UTF-8 inválido se rechazan antes de clasificar el contenido. Esos casos producen ERROR, habitualmente HTTP 400, y no generan un evento DLP en la versión actual. Un fallo interno puede producir HTTP 500. El rechazo por formato debe distinguirse del bloqueo por una política de sensibilidad.

2.2. Consultas y obtención de ejemplos

CU-02. Consultar políticas activas

El participante o evaluador consulta las reglas que explican el comportamiento del sistema. La precondición es que el servicio se encuentre disponible. Al abrir el portal se solicita GET /api/policies y se muestran identificador, nombre, descripción y puntos de las siete reglas.

La postcondición es disponer de las políticas para interpretar una decisión. La consulta no cambia los parámetros del motor ni permite editar reglas. Si la solicitud falla, la interfaz informa un error de carga.

CU-03. Consultar eventos recientes

El evaluador revisa las decisiones previas mediante GET /api/events. La consulta ocurre al abrir la página, después de una carga y al pulsar «Actualizar». Se muestran hasta 50 eventos, empezando por los más recientes, con hora, archivo, reglas, puntaje y decisión.

Si no existen eventos, se presenta una tabla vacía. La consulta no descarga documentos recibidos. El registro conserva la fecha en formato ISO UTC; la hora visible se formatea en el navegador del cliente.

CU-04. Descargar ejemplos de prueba

El participante obtiene un archivo ficticio desde los enlaces de la interfaz. Se ofrecen un JSON público, un JSON confidencial y un texto privado sin etiqueta. El servidor entrega los recursos alojados en public/ejemplos/ y el usuario puede utilizarlos posteriormente en CU-01.

La descarga del ejemplo y su posterior carga son operaciones distintas. Descargarlo no implica que el motor haya analizado una transferencia y no añade un evento DLP.

Tabla 4. Resultados de los casos de consulta

Operación	Resultado observable
Consultar políticas	Tarjetas con las reglas vigentes y sus puntajes.
Consultar eventos	Tabla de decisiones o mensaje de ausencia de registros.
Descargar ejemplo	Archivo ficticio disponible en el equipo del participante.

2.3. Operación técnica y evaluación

CU-05. Configurar y activar políticas

El operador técnico modifica los patrones o puntajes en la colección rules de app.js, o ajusta los umbrales en analyzeContent. La operación requiere acceso al código y una terminal en el equipo que ejecuta el servicio.

El flujo consiste en editar la política, comprobar su comportamiento con archivos ficticios y reiniciar el proceso Node.js. Las cargas posteriores utilizan la nueva configuración. No existe un editor web de políticas ni un registro automático de cambios administrativos.

CU-06. Evaluar el control DLP

El evaluador ejecuta npm.cmd test para comprobar los comportamientos funcionales y npm.cmd run evaluate para procesar el conjunto de 20 archivos. Ambas herramientas levantan una instancia HTTP local con directorios temporales independientes de los del portal principal.

La evaluación compara las decisiones con las etiquetas de evaluacion/casos.json, obtiene matrices de confusión y comprueba que el registro y la cantidad de archivos almacenados sean coherentes con la decisión. Una aserción incumplida o un error de ejecución interrumpe la comprobación; debe resolverse antes de usar esa salida como evidencia.

Tabla 5. Trazabilidad entre casos de uso e implementación

Caso	Elemento principal de implementación
CU-01	POST /api/upload; analyzeContent; appendEvent.
CU-02	GET /api/policies; carga de tarjetas en public/app.js.
CU-03	GET /api/events; readRecentEvents; tabla de auditoría.
CU-04	Recursos estáticos en public/ejemplos/.
CU-05	Colección rules y umbrales en app.js; reinicio manual.
CU-06	test/app.test.js y evaluacion/evaluar.js.

La configuración y la evaluación pertenecen al procedimiento técnico del laboratorio. El diagrama las representa fuera del límite de interacción web para conservar la correspondencia con el código disponible.

3. Arquitectura de red

La red Host-only conecta el cliente Kali con el anfitrión Windows. El servicio expone el portal y la API en el puerto TCP 3000; la transferencia de prueba utiliza HTTP.

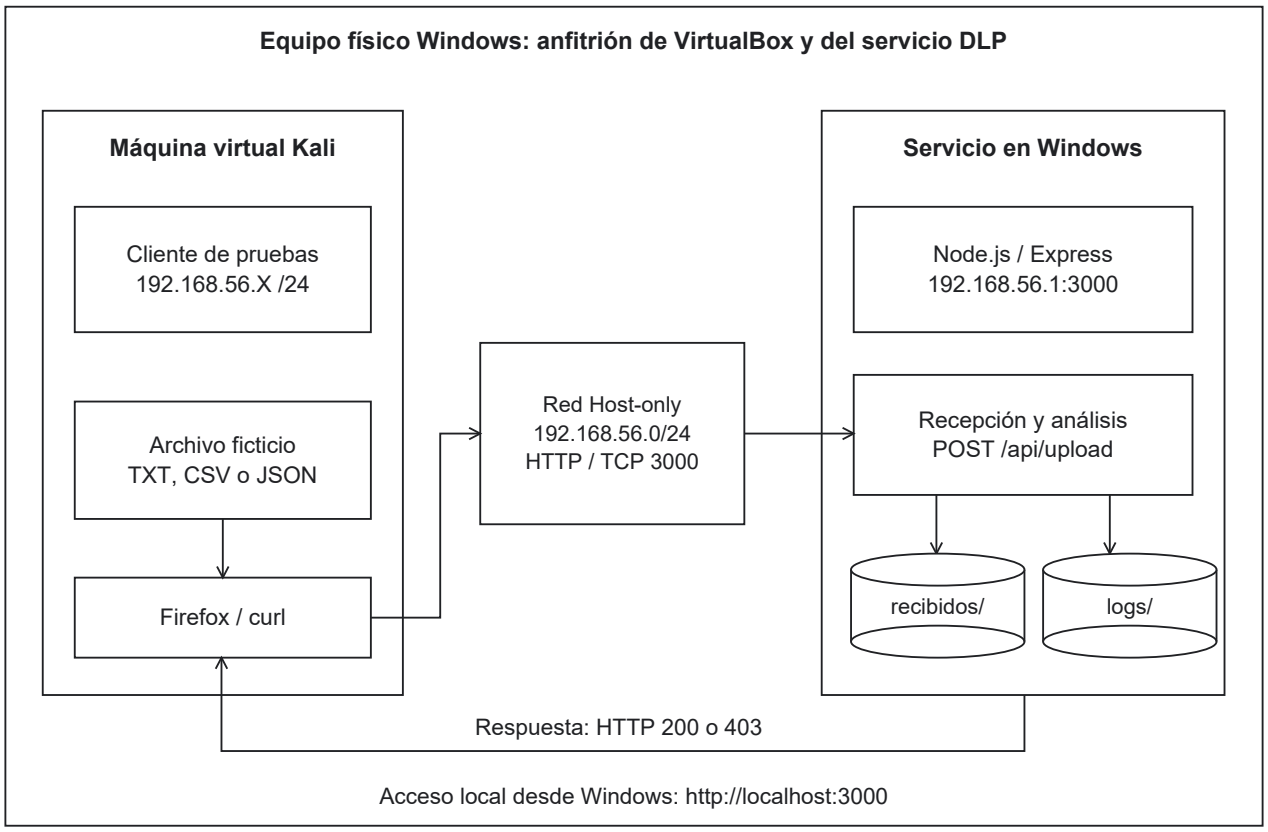


Figura 2. Despliegue del laboratorio en una computadora física.

Nota. La X representa la IP de Kali pendiente de comprobación. Fuente: configuración del laboratorio y app.js [1].

Tabla 6. Parámetros del escenario de red

Parámetro	Valor o función
Windows	192.168.56.1; dirección observada con ipconfig el 21/10/2026.
Subred	192.168.56.0/24; máscara 255.255.255.0.
Kali	Dirección propia dentro de la subred; verificar con ip -4 addr.
Destino de carga	http://192.168.56.1:3000/api/upload, desde Kali.
Escucha del servicio	0.0.0.0:3000 por defecto; configurable con HOST y PORT.

El ping comprueba conectividad ICMP, no la disponibilidad de TCP 3000. La escucha 0.0.0.0 abarca las interfaces IPv4 del equipo y no equivale a una restricción exclusiva a Host-only. La IP de Kali, el estado de la VM y el firewall no se verificaron en la revisión documental.

4. Arquitectura de software

La aplicación tiene una interfaz web y un servidor Express. Los módulos del servidor pertenecen a un mismo proceso Node.js y coordinan el análisis y la persistencia en archivos.

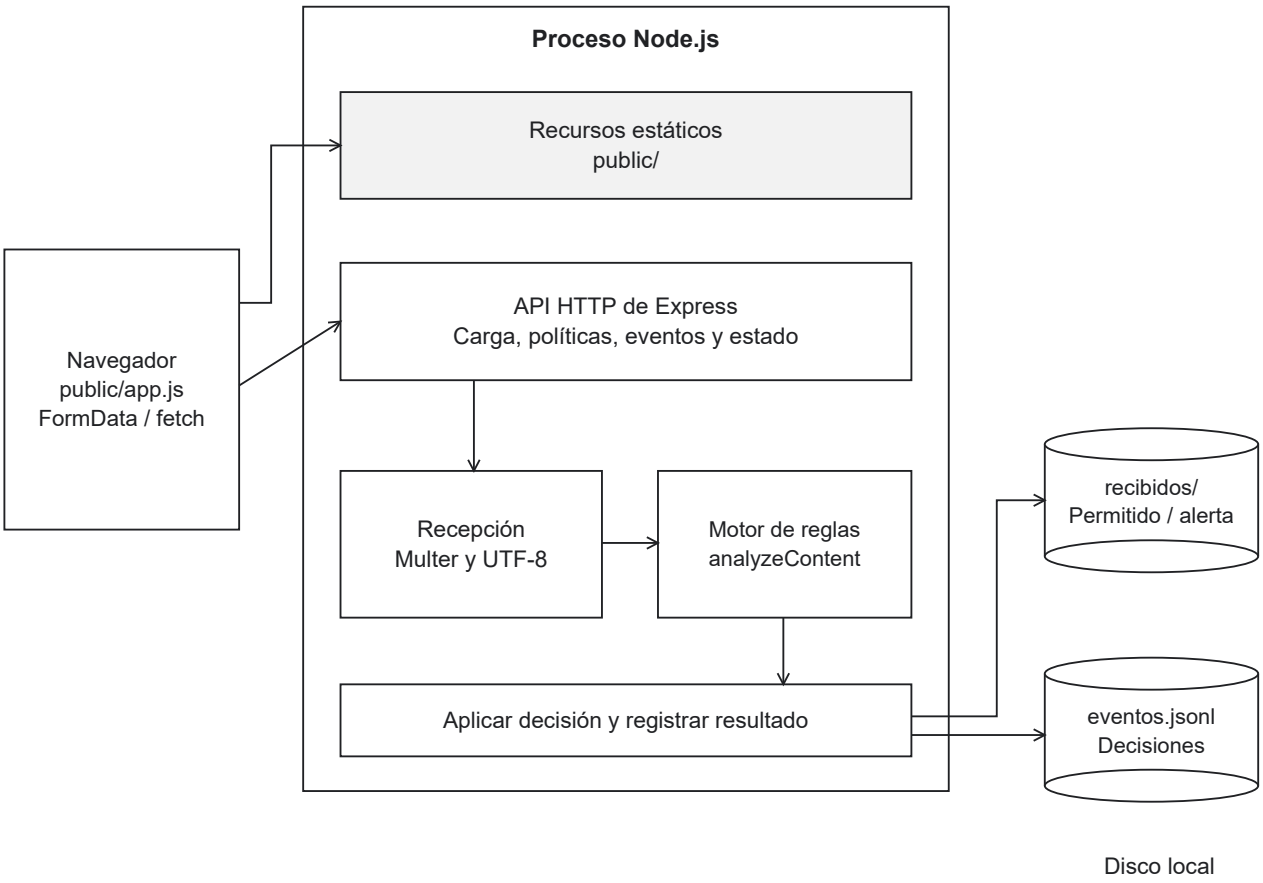


Figura 3. Componentes de software y almacenamiento local.

Nota. Las cajas representan responsabilidades lógicas, no servicios independientes. Fuente: código del prototipo [1, 2].

Tabla 7. Correspondencia de componentes con el código

Responsabilidad	Implementación
Interfaz	HTML y CSS; public/app.js construye FormData y consume la API.
Validación	Multer, comprobación de presencia y TextDecoder para UTF-8.
Análisis	rules, parseCsvRows, csvFieldsAsText y analyzeContent.
Persistencia	UUID, normalización de nombre y escritura en recibidos/.
Auditoría	appendEvent y readRecentEvents sobre eventos.jsonl.

Los JSON se examinan como texto; el motor no exige una estructura validada mediante JSON.parse. Para CSV se recorren las columnas y se agrega una representación campo: valor. El sistema no necesita una base de datos ni un servicio externo para clasificar con las reglas actuales.

5. Políticas de clasificación y decisión

La sensibilidad se aproxima mediante indicadores textuales. Cada regla coincidente aporta sus puntos una sola vez por archivo, aunque el patrón aparezca varias veces. El total expresa una política de decisión; no es una probabilidad de que el documento sea privado.

Tabla 8. Reglas y puntajes de la implementación

Regla	Indicador examinado	Puntos
DLP-001	Campo CLASIFICACION con valor CONFIDENCIAL, admitiendo comillas como en JSON.	3
DLP-002	Campo CLASIFICACION con valor RESTRINGIDO.	5
DLP-003	Campo CODIGO_ESTUDIANTE con ocho dígitos; incluye columnas CSV normalizadas.	2
DLP-004	Patrón de NOTA_FINAL con un valor entre 0 y 20; incluye columnas CSV normalizadas.	2
DLP-005	Correo del dominio ficticio universidad.edu.pe.	1
DLP-006	Etiqueta CONTRASEÑA_TEMPORAL: o CONTRASENA_TEMPORAL:.	3
DLP-007	Presencia conjunta de estudiante/alumno con nombre y nota/calificación final en el texto.	3

Tabla 9. Umbrales y efecto sobre el almacenamiento

Puntaje	Decisión	Archivo	HTTP
0	PERMITIDO	Se guarda	200
1 a 2	ALERTA	Se guarda	200
3 o más	BLOQUEADO	No se guarda	403

Ejemplo de aplicación

Un JSON con los campos `"codigo_estudiante":"20261234"` y `"nota_final":18` activa DLP-003 y DLP-004. El puntaje total es cuatro y el archivo se bloquea, aunque no incluya una clasificación confidencial.

DLP-007 comprueba que ambos patrones estén presentes en el contenido, pero no acredita que correspondan a la misma persona o frase. Las reglas tampoco verifican identidades contra un padrón institucional. Estas limitaciones explican por qué la evaluación debe incluir redacciones distintas y documentos públicos que citan ejemplos.

6. Flujo de procesamiento

La validación precede a la clasificación. Cuando la entrada es válida, las tres decisiones convergen en el registro del resultado antes de responder al cliente.

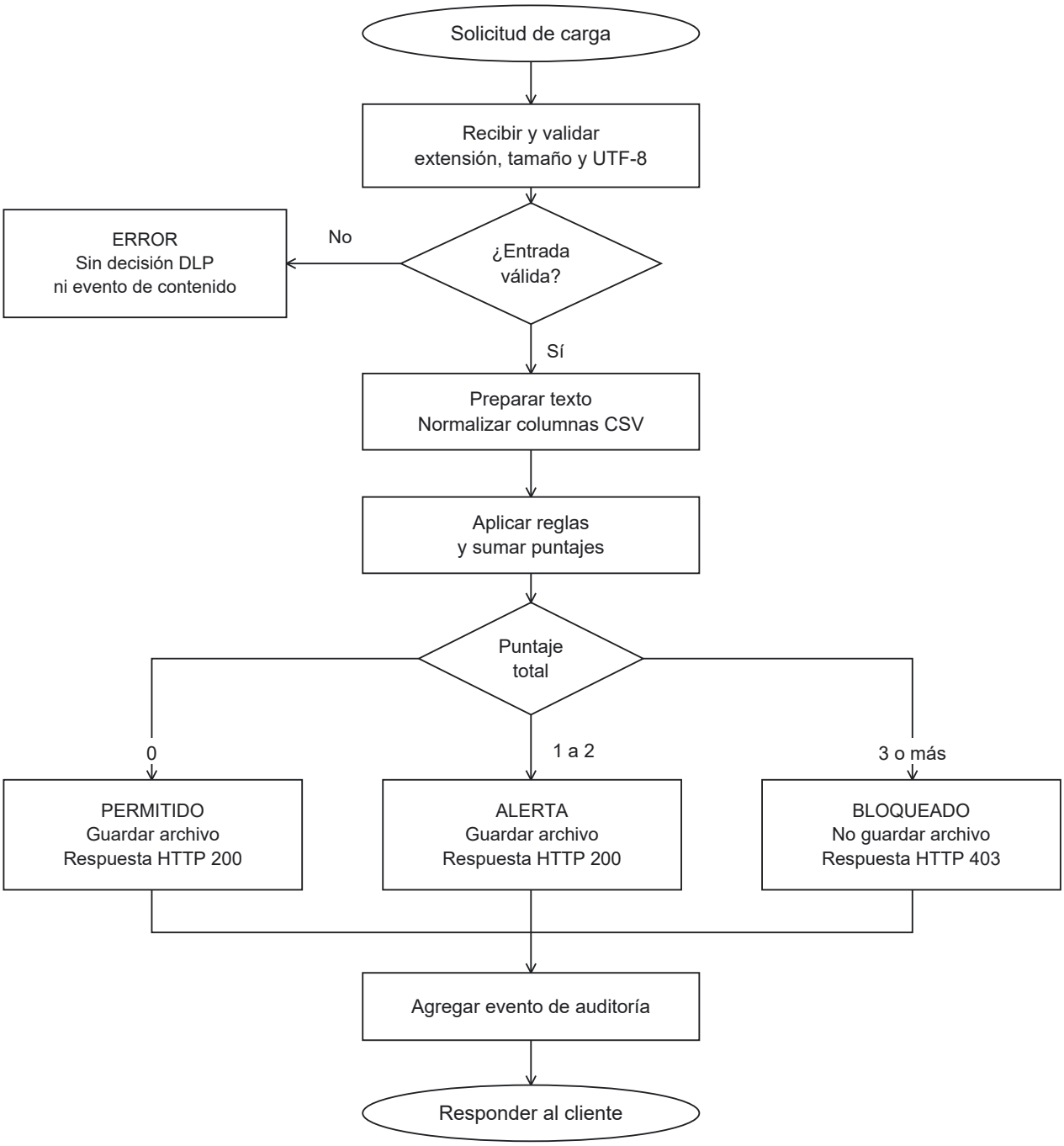


Figura 4. Flujo de validación, análisis y aplicación de la decisión.

Nota. Se representa una ejecución sin fallos internos de escritura. Fuente: POST /api/upload y manejador de errores [1].

ALERTA permite almacenar y deja la decisión visible para revisión. BLOQUEADO omite la escritura del documento. Los errores previos al análisis no aparecen en el archivo de eventos DLP actual.

7. Secuencia y persistencia

La secuencia muestra el orden de las operaciones de una carga válida y la consulta posterior que actualiza los eventos del portal.

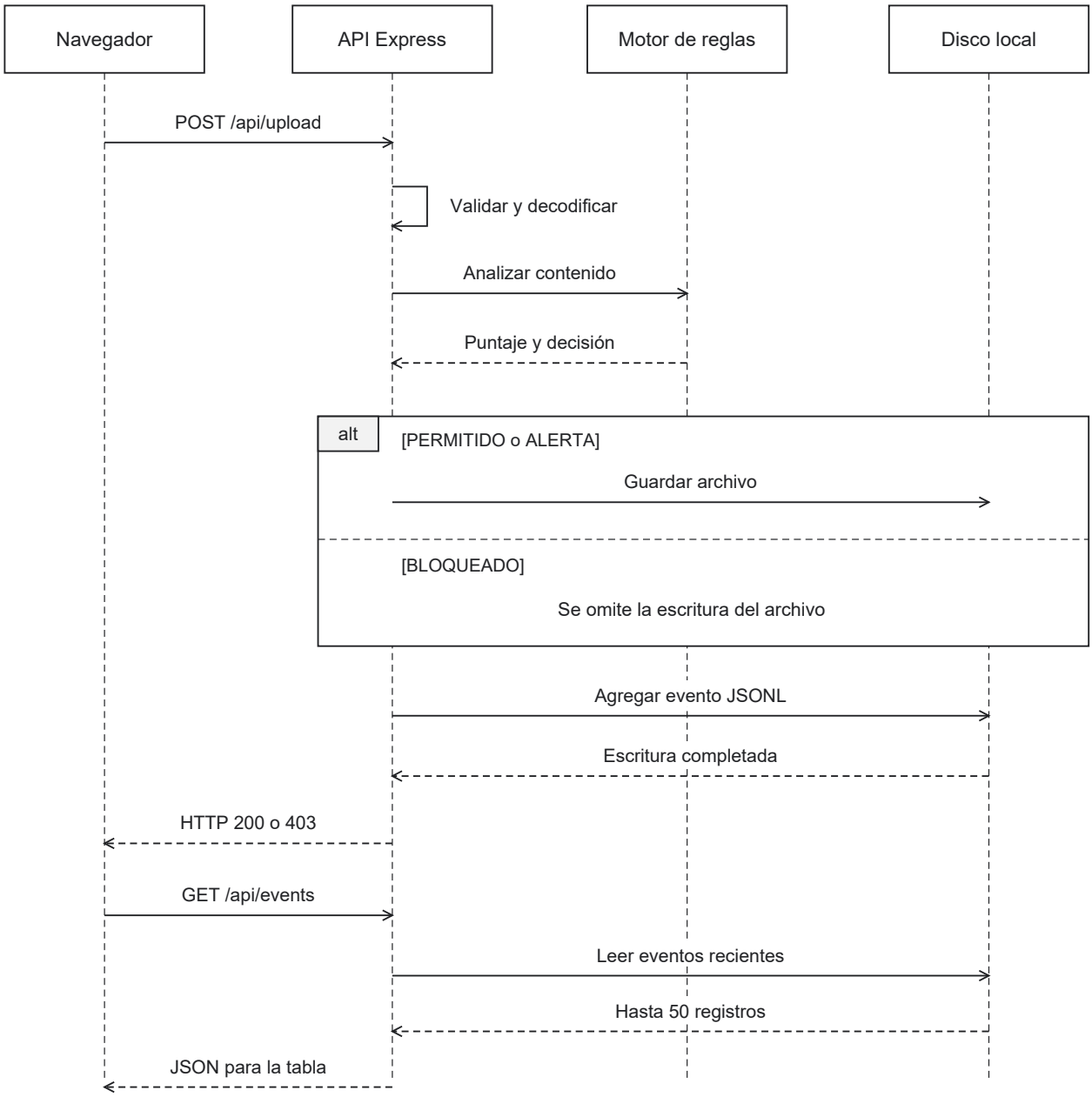


Figura 5. Interacción del cliente, la API, el motor y el disco local.

Nota. El fragmento alt distingue las alternativas de almacenamiento. Fuente: servidor y cliente JavaScript [1, 2].

Consistencia de los registros

El archivo permitido o en alerta se guarda antes de agregar el evento. Las escrituras no son atómicas: un fallo de auditoría puede dejar un archivo almacenado con una respuesta de error. En un bloqueo, storedName es nulo.

El evento conserva identificador, fecha, IP del cliente, nombre del archivo, nombre almacenado, tamaño, puntaje, coincidencias y decisión. No contiene una copia completa del documento.

8. Evaluación y lectura de resultados

8.1. Diseño de la comprobación

El repositorio incluye 20 archivos ficticios: diez públicos y diez sensibles. La clasificación de referencia está declarada en evaluacion/casos.json según el propósito y contenido del archivo, de forma independiente del puntaje que produce el motor. La batería automatizada se ejecuta por HTTP local en Windows; no debe describirse como 20 cargas originadas desde Kali.

Para medir detección se considera positivo ALERTA o BLOQUEADO. Para medir prevención solo se considera positivo BLOQUEADO, porque una alerta permite guardar. La herramienta también contrasta el número de eventos y archivos almacenados con las decisiones emitidas [3].

Tabla 10. Resultados del conjunto de 20 casos del repositorio

Medición	VP	FP	VN	FN	Precisión	Recall
Detección	10	2	8	0	83,3 %	100,0 %
Prevención	9	1	9	1	90,0 %	90,0 %

VP: verdadero positivo; FP: falso positivo; VN: verdadero negativo; FN: falso negativo. Precisión = VP / (VP + FP). Recall = VP / (VP + FN). Fuente: informe de evaluación incluido en el proyecto [3].

8.2. Interpretación de los errores

Los casos B09 y B10 son públicos, pero contienen una etiqueta de clasificación citada con fines didácticos o un correo ficticio de ejemplo. Producen falsos positivos de detección. Solo B09 se bloquea, por lo que constituye el falso positivo de prevención.

S09 corresponde a un directorio interno que activa únicamente la regla de correo institucional. El sistema lo detecta con una alerta, pero permite guardarlo; por ello es un falso negativo de prevención. El texto de notas sin etiqueta, S10, activa DLP-007 en la versión documentada.

8.3. Validez de la evidencia

Las reglas se ajustaron utilizando este mismo conjunto. Sus resultados describen una comprobación de desarrollo y no una prueba independiente de generalización. El recall de 100 % en detección no demuestra reconocimiento perfecto en documentos nuevos. Una evaluación posterior debe reservar archivos no usados al diseñar las reglas y conservar las mismas etiquetas de referencia al comparar alternativas.

9. Conclusiones, limitaciones y fuentes

9.1. Conclusiones del diseño

El prototipo permite observar una cadena completa de control: recepción, validación, clasificación, decisión, almacenamiento condicional y auditoría. La separación entre detección y prevención facilita explicar por qué una alerta no equivale a detener una transferencia. La correspondencia entre casos de uso, componentes y endpoints hace verificable el alcance de la implementación.

9.2. Limitaciones y siguiente ampliación

El alcance se limita a TXT, CSV y JSON de hasta 1 MiB, codificados como texto UTF-8 y enviados por el portal o su API. No se inspeccionan otros canales de salida ni se realiza análisis antivirus. La aplicación carece de autenticación, cuarentena y un proceso de aprobación manual; también debe mejorarse el registro de entradas inválidas y la consistencia entre archivo y auditoría.

La incorporación de Groq se plantea como una ampliación para clasificar contexto. A la fecha de este documento no forma parte del flujo. Su evaluación deberá comparar las decisiones con las del motor por reglas, usando casos nuevos y midiendo errores, tiempo de respuesta y comportamiento ante indisponibilidad.

9.3. Fuentes de la implementación

Las figuras y especificaciones se elaboraron a partir de los siguientes materiales del propio proyecto. Las referencias documentan el sistema observado y no sustituyen una revisión bibliográfica sobre DLP.

[1] Proyecto DLP. (2026). *app.js* [Código fuente]. Recepción de cargas, reglas, umbrales, persistencia y endpoints. Repositorio: https://github.com/Brando-01/DLP.

[2] Proyecto DLP. (2026). *public/index.html y public/app.js* [Código fuente]. Interfaz de carga, presentación de resultados y consulta de eventos.

[3] Proyecto DLP. (2026). *evaluacion/casos.json, evaluacion/evaluar.js y evaluacion/INFORME.md* [Materiales de evaluación]. Conjunto ficticio, procedimiento y resultados registrados.

[4] Proyecto DLP. (2026). *test/app.test.js* [Pruebas de software]. Comprobaciones funcionales del analizador y del endpoint de carga.

[5] Proyecto DLP. (2026). *README.md* [Documentación técnica]. Instalación, ejecución y configuración del laboratorio Windows / Kali.

Revisión documental: 21 de octubre de 2026. Los parámetros de red se describen según la configuración y la evidencia disponibles para esa fecha.